



INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL, I. P.
DELEGAÇÃO REGIONAL DO NORTE
CENTRO DE EMPREGO E FORMAÇÃO PROFISSIONAL DE BRAGA

Technical Report – Robot Arm and Joystick Menu

T3 - Specialist Technician in Automation,
Robotics, and Industrial Control

Bruna Ferreira Gomes Madeira



Cofinanciado pela
União Europeia

Os Fundos Europeus mais próximos de si.



Analogue Joystick Input

Joystick axes use analogue inputs because they are based on potentiometers that generate a continuous voltage signal rather than discrete ON/OFF states. This characteristic enables the system to detect not only the direction of movement, but also the magnitude of the joystick displacement.

In Arduino-based systems, the analogue-to-digital converter (ADC) operates with a 10-bit resolution, producing digital values ranging from 0 to 1023. When the joystick is in its neutral (centered) position, the potentiometers are typically at mid-travel, generating approximately 2.5 V. This voltage corresponds to a digital value close to 512.

Deadzone Concept

A deadzone is a predefined range of analogue values around the joystick's center position that is intentionally ignored by the control logic. The implementation of a deadzone is necessary because joysticks do not return precisely to their mechanical center due to manufacturing tolerances, wear, and electrical noise.

Without a deadzone, minor voltage fluctuations around the center position could be interpreted as intentional movement, resulting in jitter, drift, or unstable system behavior. By defining a deadzone, the system ensures stable and predictable control when the joystick is released.

Short Press vs. Long Press Detection

The software distinguishes between a short button press and a long button press by measuring the duration for which the button remains pressed. This is typically implemented using the `millis()` function to track elapsed time.

If the press duration is below a predefined threshold, the input is classified as a short press. If the duration exceeds this threshold, it is classified as a long press.

Button debouncing is required because mechanical switches do not generate a clean digital signal when actuated. Instead, they produce rapid oscillations between logical states during press and release events. Without debouncing, these transitions could be



misinterpreted as multiple presses. Debouncing ensures that each physical action is registered as a single, valid input.

Servo Control Limits

Servo motors must be operated within a control range of 0° to 180° , which corresponds to their physical and electrical limits. Commands outside this range may force the servo to attempt movement beyond its mechanical stops.

Ignoring these limits can cause the servo to stall, draw excessive current, overheat, or experience mechanical stress, potentially leading to permanent damage. Additionally, excessive current consumption can introduce power instability that negatively affects the microcontroller and other connected components.

LCD Interface and Communication Mode

Using a 4-bit parallel interface instead of an 8-bit interface significantly reduces the number of required GPIO pins. This is particularly advantageous in Arduino projects, where available pins are limited and often shared with other peripherals such as joysticks, servos, and buttons.

Although 4-bit mode results in slightly slower data transmission, this performance reduction is negligible for typical LCD text updates.

In this project, an I²C LCD shield was also used. The I²C interface further minimizes wiring complexity by reducing the number of required connections to only two communication lines (SDA and SCL), in addition to power and ground. This simplifies circuit assembly, improves reliability, and frees additional pins for other functions.

Power Integrity Considerations

Servo motors can cause unintended Arduino resets due to their high peak current consumption, particularly during startup or when operating under mechanical load. These current transients can result in temporary voltage drops that are sufficient to reset the microcontroller.

Two primary mitigation strategies are generally recommended:



- Powering the servos using a dedicated external power supply while maintaining a common ground reference with the Arduino.
- Adding decoupling capacitors near the servo power lines to absorb transient current spikes and stabilize the supply voltage.

In this implementation, a 100 μF decoupling capacitor was used. While this capacitor provides a degree of voltage smoothing and helps reduce minor fluctuations, its capacitance value is relatively low for handling the high inrush currents typically generated by servo motors. As a result, it offers limited protection against larger current spikes compared to higher-value capacitors (commonly 470 μF or above).

Nevertheless, the 100 μF capacitor contributes to improved power stability and demonstrates the importance of local energy storage in mitigating transient disturbances, even if it does not fully eliminate the risk of voltage drops under heavy load conditions.

Base System Code

```
#include <Arduino.h>
#include <Servo.h>
#include <LiquidCrystal_I2C.h>

// CONFIGURAÇÃO DO LCD
LiquidCrystal_I2C lcd(0x27,16,2);

// ----- Joystick -----
#define JOY_X    A0    // HORZ
#define JOY_Y    A1    // VERT
#define JOY_SEL  2     // clique (INPUT_PULLUP)
// ----- Servos (2 eixos) -----
Servo servoV;
// Axis 1 (Vertical)
Servo servoH;
// Axis 2 (Horizontal)
#define SERVO_V_PIN 6
#define SERVO_H_PIN 5
// ----- Parâmetros -----
int angV = 90;
int angH = 90;
const int ANG_MIN = 0;
const int ANG_MAX = 180;
const int ADC_CENTER = 512;
const int DEADZONE    = 120;
const unsigned long REPEAT_MS_MIN = 50;
```



```
const unsigned long REPEAT_MS_MAX = 220;
const unsigned long LONGPRESS_MS = 800;
const unsigned long SEL_DB_MS = 40;
// ----- Menu / Estados -----
enum Mode {
MODE_MENU = 0,
MODE_MENU = 0,
MODE_AXIS1_VERT,
MODE_AXIS2_HORZ
};
Mode mode = MODE_MENU;
int menuIndex = 0;
unsigned long tRepeat = 0;
// LCD anti-flicker
int lastLine0Hash = -1;
int lastLine1Hash = -1;
// Botão SEL
bool selLastRaw = HIGH;
bool selStable = HIGH;
unsigned long tDebounce = 0;
unsigned long tPressStart = 0;
bool longPressFired = false;
int hashLine(const char* s) {
int h = 0;
while (*s) h = (h * 31) + *s++;
return h;
}
void lcdWrite2(const char* l0, const char* l1) {
int h0 = hashLine(l0);
int h1 = hashLine(l1);
if (h0 == lastLine0Hash && h1 == lastLine1Hash) return;
lastLine0Hash = h0;
lastLine1Hash = h1;
lcd.setCursor(0, 0);
lcd.print(" ");
lcd.setCursor(0, 0);
lcd.print(l0);
lcd.setCursor(0, 1);
lcd.print(" ");
lcd.setCursor(0, 1);
lcd.print(l1);
}

int joyAxisStep(int pin, int &magAbs) {
int v = analogRead(pin);
int delta = v - ADC_CENTER;
if (delta > DEADZONE) { magAbs = delta; return +1; }
```



```
if (delta < -DEADZONE){ magAbs = -delta; return -1; }
magAbs = 0;
return 0;
}

unsigned long intervalByMag(int magAbs) {
    if (magAbs < 0) magAbs = 0;
    if (magAbs > 511) magAbs = 511;
    return (unsigned long)map(magAbs, 0, 511, REPEAT_MS_MAX,
REPEAT_MS_MIN);
}

bool allowRepeat(unsigned long intervalMs) {
    unsigned long now = millis();
    if (now - tRepeat >= intervalMs) {
        tRepeat = now;
        return true;
    }
    return false;
}

// SEL: 1=short click, 2=long press, 0=none
int selEvent() {
    bool raw = digitalRead(JOY_SEL);

    if (raw != selLastRaw) {
        selLastRaw = raw;
        tDebounce = millis();
    }

    if (millis() - tDebounce > SEL_DB_MS) {
        if (raw != selStable) {
            selStable = raw;

            if (selStable == LOW) {
                tPressStart = millis();
                longPressFired = false;
            } else {
                if (!longPressFired) return 1;
            }
        }
    }

    if (selStable == LOW && !longPressFired) {
        if (millis() - tPressStart >= LONGPRESS_MS) {
            longPressFired = true;
            return 2;
        }
    }
}
```



```
    return 0;
}

void drawMenu() {
    const char* items[3] = {
        "1) Axis1 VERT",
        "2) Axis2 HORZ",
        "3) Exit"
    };

    char line0[17];
    char line1[17];

    snprintf(line0, sizeof(line0), "Menu: %d/3", menuIndex + 1);
    snprintf(line1, sizeof(line1), "%s", items[menuIndex]);

    lcdWrite2(line0, line1);
}

void drawAxis1() {
    char l0[17], l1[17];
    snprintf(l0, sizeof(l0), "Axis1 VERT");
    snprintf(l1, sizeof(l1), "Ang:%3d Hold=EXIT", angV);
    lcdWrite2(l0, l1);
}

void drawAxis2() {
    char l0[17], l1[17];
    snprintf(l0, sizeof(l0), "Axis2 HORZ");
    snprintf(l1, sizeof(l1), "Ang:%3d Hold=EXIT", angH);
    lcdWrite2(l0, l1);
}

void setup() {
    pinMode(JOY_SEL, INPUT_PULLUP);

    servoV.attach(SERVO_V_PIN);
    servoH.attach(SERVO_H_PIN);
    servoV.write(angV);
    servoH.write(angH);

    lcd.begin(16, 2);
    lcd.init();
    lcd.backlight();
    lcd.clear();
    lcdWrite2("Joystick + Servo", "Robot Arm Ready");
    delay(1200);
}
```



```
mode = MODE_MENU;
menuIndex = 0;
drawMenu();
}

void loop() {
    int ev = selEvent();

    if (mode == MODE_MENU) {
        int mag = 0;
        int yStep = joyAxisStep(JOY_Y, mag);
        unsigned long rep = intervalByMag(mag);

        // yStep>0 (cima) -> menuIndex--
        // yStep<0 (baixo) -> menuIndex++
        if (yStep != 0 && allowRepeat(rep)) {
            if (yStep > 0) {
                if (menuIndex > 0) menuIndex--;
            } else {
                if (menuIndex < 2) menuIndex++;
            }
            drawMenu();
        }

        if (ev == 1) {
            if (menuIndex == 0) { mode = MODE_AXIS1_VERT; drawAxis1(); }
            else if (menuIndex == 1) { mode = MODE_AXIS2_HORZ; drawAxis2(); }
            else { menuIndex = 0; drawMenu(); }
        }
    }

    if (mode == MODE_AXIS1_VERT) {
        if (ev == 2) { mode = MODE_MENU; drawMenu(); delay(150); return; }

        int mag = 0;
        int yStep = joyAxisStep(JOY_Y, mag);
        unsigned long rep = intervalByMag(mag);

        if (yStep != 0 && allowRepeat(rep)) {
            if (yStep > 0) angV++;
            else angV--;

            if (angV < ANG_MIN) angV = ANG_MIN;
            if (angV > ANG_MAX) angV = ANG_MAX;

            servoV.write(angV);
            drawAxis1();
        }
    }
}
```



```
}  
}  
  
if (mode == MODE_AXIS2_HORZ) {  
    if (ev == 2) { mode = MODE_MENU; drawMenu(); delay(150); return; }  
  
    int mag = 0;  
    int xStep = joyAxisStep(JOY_X, mag);  
    unsigned long rep = intervalByMag(mag);  
  
    if (xStep != 0 && allowRepeat(rep)) {  
        // Convenção: um lado aumenta, outro diminui  
        if (xStep > 0) angH++;  
    else angH--;  
    if (angH < ANG_MIN) angH = ANG_MIN;  
    if (angH > ANG_MAX) angH = ANG_MAX;  
    servoH.write(angH);  
    drawAxis2();  
    }  
    }  
    delay(10);  
}
```

Simulation Link: Simulation – [Base System on Wokwi](#)